

SECTION A - PYTHON + OPENCV CODE REFERENCE

Imports: import cv2, numpy as np, matplotlib.pyplot as plt, pandas as pd, pywt (skimage for hog/LBP). **Colors are BGR:** red (0,0,255) green (0,255,0) blue (255,0,0) yellow (0,255,255).

A1 Basics, drawing, blending, masking (Lab 1)

```
# BGR; returns None if path wrong!
img = cv2.imread(p)
g = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # needed for matplotlib
# shape = (rows, cols, channels)
h, w = img.shape[:2]
img[y, x]      img[y1:y2, x1:x2]          # index = [row, col] = [y, x]
cv2.resize(img, (w2, h2))                 # dsize = (WIDTH, HEIGHT)
cv2.resize(img, None, fx=.5, fy=.5, interpolation=cv2.INTER_AREA)
cv2.GaussianBlur(img, (5,5),0)            cv2.blur(img,(5,5))    cv2.medianBlur(img,5)
cv2.line(img, (x1,y1), (x2,y2), col, th)  # drawing uses (x, y)
cv2.rectangle(img, (x1,y1), (x2,y2), col, th) # th=-1 -> filled
cv2.circle(img, (cx,cy), r, col, -1)
# (x,y)=bottom-left
cv2.putText(img, "t", (x,y), cv2.FONT_HERSHEY_SIMPLEX, 1, col, 2)
# saturates at 255 (np a+b WRAPS)
cv2.add(a,b)
# a1*a + b e*b ; same size/channels
cv2.addWeighted(a,a1,b,b,e,0)
cv2.bitwise_and(a,a,mask=m)    cv2.bitwise_or(a,b)    cv2.bitwise_not(m)
b,g_,r = cv2.split(img); img2 = cv2.merge([b,g_,r])
ret, bw = cv2.threshold(g,127,255,cv2.THRESH_BINARY) # TWO return values
canvas = np.zeros((800,800,3), np.uint8)
df = pd.DataFrame(img.reshape(-1,3), columns=['B','G','R']); df.describe()
plt.subplot(r,c,i); plt.imshow(x, cmap='gray'); plt.axis('off'); plt.title(t)
# imshow wants BGR
cv2.imshow(n,x); cv2.waitKey(0); cv2.destroyAllWindows()
```

A2 Photometric transforms (Lab 2)

```
neg = 255 - g
gf = g.astype(np.float32)          # CAST first (uint8 wraps)
log_img = np.uint8(255/np.log(1+gf.max())) * np.log(1+gf))
lut = np.array([255*(i/255.0)**gam for i in range(256)], np.uint8)
out = cv2.LUT(g, lut)              # gamma<1 brighter, >1 darker
out = np.interp(g, [0,80,170,255], [0,30,225,255]).astype(np.uint8)
cv2.normalize(g, None, 0, 255, cv2.NORM_MINMAX) # min-max stretch
hist = cv2.calcHist([g],[0],None,[256],[0,256]); plt.hist(g.ravel(),256,[0,256],1)
eq = cv2.equalizeHist(g)           # 8-bit SINGLE channel only
cl = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)).apply(g)
y = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb); y[:, :, 0] = cv2.equalizeHist(y[:, :, 0])
col = cv2.cvtColor(y, cv2.COLOR_YCrCb2BGR) # equalize COLOR images via Y channel
heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET) # needs 1-channel 8-bit
ct3 = cv2.cvtColor(ct, cv2.COLOR_GRAY2BGR) # match channels before addWeighted
fused = cv2.addWeighted(ct3, 0.7, heat, 0.3, 0)
# video: order = equalize -> colormap -> balance -> log/gamma
cap = cv2.VideoCapture('v.mp4')
while True:
    ret, f = cap.read()
    if not ret: break
    cv2.imshow('o', np.hstack([f, proc])) # same height AND channels
    if cv2.waitKey(30) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

A3 Geometric transforms (Lab 3)

```
# translate
M = np.float32([[1,0,tx],[0,1,ty]]); cv2.warpAffine(img, M, (w,h))
# +ang = counter-clockwise
M = cv2.getRotationMatrix2D((w/2,h/2), ang, scale)
cv2.warpAffine(img, M, (w,h)) # fixed canvas -> corners cut
cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE) # exact 90 turns, no loss
M = np.float32([[1,k,0],[0,1,0]]) # shear
# 1 = left-right, 0 = up-down, -1 both
cv2.flip(img, 1)
M = cv2.getAffineTransform(src3, dst3) # 3 pairs, float32 (3,2) -> 2x3
M = cv2.getPerspectiveTransform(src4, dst4) # 4 pairs, float32 (4,2) -> 3x3
# 3x3 here; warpAffine takes 2x3
cv2.warpPerspective(img, M, (W,H))
M3 = T @ R # rightmost matrix acts FIRST
cv2.warpAffine(img, M3[:2:], (w,h)) # pass top 2 rows of a 3x3
```

A4 Filtering, edges, features (Lab 4)

```
cv2.filter2D(img, -1, k) # correlation, SAME size output
cv2.filter2D(img, cv2.CV_64F, k) # keep signed values
k = np.ones((3,3), np.float32)/9 # box blur (sums to 1)
sharp = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]], np.float32) # sums to 1
usm = cv2.addWeighted(img, 1.5, blur, -0.5, 0) # unsharp mask
gx = cv2.Sobel(g, cv2.CV_64F, 1, 0, ksize=3) # dx: VERTICAL edges
# dy: HORIZONTAL edges
gy = cv2.Sobel(g, cv2.CV_64F, 0, 1, ksize=3)
mag = np.sqrt(gx**2 + gy**2); ang = np.arctan2(gy, gx)
mag8 = np.uint8(np.clip(mag, 0, 255)); cv2.Scharr(g, cv2.CV_64F, 1, 0)
lap = cv2.convertScaleAbs(cv2.Laplacian(cv2.GaussianBlur(g,(5,5),1.4), cv2.CV_64F))
# Canny does NOT blur
edges = cv2.Canny(cv2.GaussianBlur(g,(5,5),1.4), 50, 150)
from skimage.feature import hog, local_binary_pattern
feat, himg = hog(g, orientations=9, pixels_per_cell=(8,8),
                  cells_per_block=(2,2), visualize=True)
lbp = local_binary_pattern(himg, P=8, R=1, method='uniform') # values 0..9
hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0,11), range=(0,10))
# color histogram
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
ch = cv2.calcHist([hsv],[0,1,2],None,[8,8,8],[0,180,0,256,0,256])
# local std (texture contrast): sqrt(E[x^2] - E[x]^2)
f = g.astype(np.float32); kk = np.ones((5,5), np.float32)/25
m = cv2.filter2D(f, -1, kk); m2 = cv2.filter2D(f*f, -1, kk)
contrast = np.sqrt(np.maximum(m2 - m**2, 0))
```

A5 Segmentation (Lab 5)

```
# BINARY_INV TRUNC TOZERO TOZERO_INV
_, m = cv2.threshold(g, T, 255, cv2.THRESH_BINARY)
# T returned
T, m = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
m = cv2.adaptiveThreshold(g, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                          # ONE return; blockSize odd
                          cv2.THRESH_BINARY, 11, 2)
# adaptive: T(x,y) = local_mean - C ; larger C -> lower T -> MORE white
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV) # H 0-179, S,V 0-255
m = cv2.inRange(hsv, (20,100,100), (35,255,255)) # yellow
m = cv2.bitwise_or(cv2.inRange(hsv, (0,100,80), (10,255,255)),
                  # RED wraps
                  cv2.inRange(hsv, (170,100,80), (179,255,255)))
obj = cv2.bitwise_and(img, img, mask=m)
k = np.ones((5,5), np.uint8)
# erode->dilate: kills white specks
cv2.morphologyEx(m, cv2.MORPH_OPEN, k)
# dilate->erode: fills black holes
cv2.morphologyEx(m, cv2.MORPH_CLOSE, k)
otsu, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
# K-Means colour segmentation
px = np.float32(img.reshape(-1,3))
crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
_, lab, cen = cv2.kmeans(px, K, None, crit, 10, cv2.KMEANS_RANDOM_CENTERS)
seg = np.uint8(cen)[lab.flatten()].reshape(img.shape)
# watershed: thresh(INV+OTSU) -> OPEN -> sure_bg-dilate -> distanceTransform
# -> sure_fg=thresh(dist, frac*max) -> unknown=bg-fg -> connectedComponents
# -> markers+1, markers[unknown==255]=0 -> cv2.watershed(img_bgr, markers)
# boundaries = -1, background = 1, objects = 2.. (full code: C5)
```

A6 Hough, SIFT, stitching, wavelets (Lab 6)

```
# Hough needs EDGES
edges = cv2.Canny(cv2.GaussianBlur(g,(5,5),0), 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=50,
                       # None if nothing
                       minLineLength=50, maxLineGap=10)
for x1,y1,x2,y2 in segs[:,0]: cv2.line(img, (x1,y1), (x2,y2), (0,255,0), 2)
# (rho, theta) pairs
lines = cv2.HoughLines(edges, 1, np.pi/180, 150)
circ = cv2.HoughCircles(cv2.medianBlur(g,5), cv2.HOUGH_GRADIENT, dp=1,
                        minDist=30, param1=100, param2=30, minRadius=10, maxRadius=60)
# circ shape (1,N,3) -> count = circ.shape[1]; check None first
# des (N,128)
sift = cv2.SIFT_create(); kp, des = sift.detectAndCompute(g, None)
img_kp = cv2.drawKeypoints(img, kp, None,
                           flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
bf = cv2.BFMatcher(cv2.NORM_L2); mt = bf.knnMatch(des1, des2, k=2)
good = [m for m,n in mt if m.distance < 0.75*n.distance] # ratio test
src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape((-1,1,2))
dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape((-1,1,2))
# H: img1 -> img2
H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
# corners (4,1,2) f32
box = cv2.perspectiveTransform(corners, H)
cv2.polylines(scene, [np.int32(box)], True, (0,255,0), 3)
cv2.drawMatches(i1, kp1, i2, kp2, good, None, flags=2)
cA, (cH,cV,cD) = pywt.dwt2(g, 'haar'); pywt.idwt2((cA,(cH,cV,cD)), 'haar')
co = pywt.waverec(sig, 'db4', level=3); sig2 = pywt.waverec(co, 'db4')
pywt.threshold(c, thr, mode='soft')
```

SECTION B - CONCEPT NOTES AND HAND CALCULATIONS

B1 Basics (Lab 1)

Image = array (rows, cols, ch) uint8. **BGR** in OpenCV, RGB in matplotlib. Index [y,x], draw (x,y).
Gray for: threshold, edges, Hough, hist-eq, SIFT. **Color** for: HSV segmentation, K-Means. **add vs +**: 200+100 = 255 (cv2.add) vs 44 (NumPy wrap). **THRESH_BINARY** is strictly greater: 127 with T=127 -> 0. Kernels must be odd. Draw text AFTER addWeighted. Mask logic: and(img1,mask) + and(img2,not mask) merged by OR. DIP steps: acquisition, enhancement, restoration, color, wavelets, compression, morphology, segmentation, representation, recognition.

B2 Photometric (Lab 2) s = T(r), no neighbours, no movement

Log s = c*log(1+r), c = 255/log(1+max). Expands darks, compresses brights (r=10 -> ~110).
Gamma s = 255*(r/255)^g. g<1 brighter, g>1 darker (64: g=.5 -> 128, g=2 -> 16). **Piecewise** linear contrast points (0,0),(80,30),(170,225),(255,255): r=125 -> 127.5, mid slope 2.17 boosts mid contrast. **Hist-eq** s = 255*CDF(r): automatic, nonlinear, global; grayscale only; color via Y of YCrCb. **CLAHE** = local tiles + clip limit (less noise amplification). **Diagnose by histogram**: left-bunched -> log/gamma<1/eq; right-bunched -> gamma>1; narrow -> stretch/eq. **JET** heatmap: blue low, red high; input 1-channel 8-bit, output 3-channel. **Pipeline order**: equalize (1ch) -> colormap (3ch) -> color balance -> log/gamma. **Gray-world balance**: scale each channel by avg/channel_mean; clip before casting. **Fusion**: equalize each, colormap one, same size/channels, weights ~ sum 1.

B3 Geometric (Lab 3) moves WHERE a pixel is

Matrix columns = where unit x and y vectors land. Scale [[sx,0],[0,sy]], shear [[1,k],[0,1]], reflect [[1,0],[0,-1]], rotate [[c,-s],[s,c]]. Linear cannot translate (A*0=0) -> **homogeneous (x,y,1)**. 3x3. Order: M = T @ R means R first. Scale then translate (5,5) of (3,4) -> (11,13); reverse -> (16,18). Rotation about center = T(c) R T(-c).

Hierarchy (DOF): translation 2, rigid 3, similarity 4, **affine 6** (keeps parallelism, collinearity, ratios), **projective 8** (keeps only straight lines). **Affine needs 3 point pairs, projective 4**. Trapezoid -> rectangle needs projective (affine keeps parallel lines parallel). warpAffine takes 2x3, warpPerspective 3x3, dsize = (w,h). 45 deg on same canvas cuts corners: scale<1 or expand canvas.

B4 Filtering, edges, features (Lab 4)

Conv = slide kernel, multiply, sum. Smoothing kernels sum to 1; edge kernels sum to 0 (flat -> 0). Output = floor((N-k+2p)/s)+1: 11,k3,p0,s2 -> 5. filter2D = correlation, same-size, no strides argument. **Sobel-x** on [10,10,200]x3 = 760. dx detects vertical edges. Use CV_64F or negatives clip. **Canny**: Gaussian -> gradient -> non-max suppression -> hysteresis (>high strong; <low gone; between keep only if touching strong). Canny does not blur: do it yourself. Laplacian = 2nd derivative, noisy; blur first (LoG), zero-crossings. **Filter choice**: Gaussian general noise, median salt-and-pepper, bilateral edge-preserving.

HOG gradients -> 8x8 cell 9-bin histograms -> 2x2 block normalise -> concat; 64x128 -> 7*15 blocks*36 = **3780**. **LBP** neighbour >= center -> 1; center 50, nbrs 60,40,55,50,20,70,10,90 -> 10110101 = **181**; lighting-invariant; uniform = P+2 = 10 bins. Texture contrast = local std. **Manual bugs**: arctan2 range -180..180 vs histogram range (0,360) -> use %360; contrast code computes mean not std; 1-D Gaussian kernel only blurs one way; strides arg not real. By hand: box blur of [10,20,30],[10,20,30],[80,90,100]] centre = 390/9 = 43.3; Sobel-y = 280.

B5 Segmentation (Lab 5)

Types T=100 on [30,100,101,200]: BINARY [0,0,255,255]; INV [255,255,0,0]; TRUNC [30,100,100,100]; TOZERO [0,0,101,200]. **Global** fails under uneven light. **Otsu** maximises between-class variance, needs bimodal histogram, T arg ignored. **Adaptive** T = local mean - C: small block = noisy/hollow, huge block = global-like, **larger C = lower T = more white** (less noise, thinner strokes); GAUSSIAN usually beats MEAN. **HSV** hue ~ lighting-independent; red needs 2 strokes; too-tight mask misses shaded pixels -> widen S,V. **Region growing**: seed + similarity (intG) casts; T too small = tiny, too large = leaks; seed changes the 'similar' range.

Watershed: distance transform cores = markers. **Raise frac** (0.2 -> 0.7): smaller cores, better separation, risk of lost or over-split objects; low frac = merged. ret from connectedComponents counts background: objects = ret-1. **K-Means**: [10,12,14,200,210], init 10,200 -> centroids 12, 205; output has exactly K colors; colour only, no position. **Assumptions**: global=uniform light; Otsu=bimodal; adaptive=local contrast; HSV=distinct hue; Canny=strong edges; region growing=homogeneous connected; watershed=blobs with distinct cores; K-Means=colour clusters = regions.

B6 Hough, wavelets, SIFT, stitching (Lab 6)

Hough line: rho = x cos(t) + y sin(t) (vertical lines break y=mx+b). Pixel (30,40): t=0 -> 30, t=90 -> 40, t=53.13 -> 50; vertical x=30 -> t=0, rho=30. Lower threshold = more (false) lines; maxLineGap joins collinear bits. **HoughCircles**: 3D (a,b,r); param2 lower = more circles + false; minDist small = double circles, large = missed neighbours; set min/max radius. Lanes: Canny + ROI mask + HoughLinesP + slope filter (y down: left lane slope < 0).

Wavelets give frequency AND location (Fourier: frequency only). DWT: approx + detail, each half length. 2D 256x256 -> four 128x128 (cA, cH, cV, cD). CWT continuous (Morlet); DWT discrete (Haar, db, sym); WPT splits details too. Denoise: shrink detail coeffs (soft), keep cA, residual = sig - denoised, flag [res] > k*sigma.

SIFT: DoG scale-space extrema -> localise (26 neighbours, reject low-contrast/edge) -> orientation (dominant gradient) -> descriptor 4x4x8 = **128**. Ratio test 0.75 rejects ambiguous matches. **RANSAC**: min 4 matches for H; fit on random minimal sets, keep most inliers. knnMatch(des1,des2): queryIdx -> kp1, trainIdx -> kp2. H maps first (src) image into second (dst) frame.

Stitching: SIFT -> match+ratio -> H (right->left) -> canvas from warped corners (shift by T) -> warp right, paste left -> crop. Fails with <20% overlap, low texture, moving objects, **parallax** (camera translated; homography needs rotation or planar scene), exposure seams, swapped src/dst. Security zone: Canny in zone mask vs empty-zone baseline.

B7 Which method when

Uniform light, clear contrast: global | unknown T, bimodal: Otsu | uneven light: adaptive Gaussian | color object: HSV inRange | touching blobs: distance + watershed | region from known point: region growing | outlines only: Canny | many colors: K-Means | lines: Canny+HoughLinesP | circles: HoughCircles | known object: SIFT+RANSAC | panorama: SIFT+H+warp | dark image: log/gamma<1 | low contrast: equalize/CLAHE | tilted page: getPerspectiveTransform | salt-and-pepper: median | sensor anomaly: wavelet residual.

B8 Gotchas that cost marks

1 BGR vs RGB. 2 [row,col] vs (x,y). 3 dsize=(w,h) but shape=(h,w). 4 uint8 wraps (+, log(1+g), uint8 subtract). 5 equalizeHist: gray only. 6 threshold returns 2 values, adaptiveThreshold 1. 7 Sobel needs CV_64F. 8 Canny does not blur. 9 kernel/blockSize odd. 10 red needs two hue ranges. 11 watersheded markers: bg=1, unknown=0, image must be 3-channel. 12 connectedComponents count includes background. 13 queryIdx/trainIdx order. 14 H direction in stitching. 15 HoughLines/HoughCircles return None. 16 np.hstack needs same shape. 17 clip before casting to uint8. 18 apply colormap AFTER equalize. 19 getAffineTransform 3 pts, perspective 4, float32. 20 draw text after blending.

SECTION C - SOLVED PRACTICE TASKS (the questions we worked)

C1 Lab 1 tasks

1A target board (600x600, radii 250/190/130/70, red-yellow, blue box):

```
canvas = np.zeros((600,600,3), np.uint8); h, w = canvas.shape[:2]
cx, cy = w//2, h//2
colors = [(0,0,255), (0,255,255)] # red, yellow
for i, r in enumerate([250,190,130,70]): # biggest first
    cv2.circle(canvas, (cx,cy), r, colors[i%2], -1)
cv2.rectangle(canvas, (cx-250,cy-250), (cx+250,cy+250), (255,0,0), 3)
print("Center:", (cx,cy))
plt.imshow(cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB)); plt.axis('off')
```

1B blur 15x15, centre 200x200 crops, side by side:

```
img = cv2.imread('a.jpg')
if img is None: print("Error: Image not found.")
else:
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    blur = cv2.GaussianBlur(rgb, (15,15), 0)
    h, w = rgb.shape[:2]; y1, x1 = h//2-100, w//2-100
    r1 = rgb[y1:y1+200, x1:x1+200]; r2 = blur[y1:y1+200, x1:x1+200]
    plt.subplot(1,2,1); plt.imshow(r1); plt.axis('off')
    plt.title('Original', fontsize=14, color='green')
    plt.subplot(1,2,2); plt.imshow(r2); plt.axis('off')
    plt.title('Blurred', fontsize=14, color='crimson'); plt.show()
```

1C transparent top banner (text AFTER blending, else it gets faded/tinted):

```
h, w = img.shape[:2]; ov = img.copy()
cv2.rectangle(ov, (0,0), (w,int(0.15*h)), (0,255,0), -1)
out = cv2.addWeighted(ov, 0.6, img, 0.4, 0)
cv2.putText(out, "Title", (20,int(0.10*h)), cv2.FONT_HERSHEY_SIMPLEX,
    1.5, (255,255,255), 3)
```

C2 Lab 2 tasks

2A gamma with LUT + 2x2 grid (histogram shifts right and spreads). Use vmin/vmax so display is not auto-stretched:

```
def gamma_correct(g, gamma):
    lut = np.array([255*(i/255.0)**gamma for i in range(256)], np.uint8)
    return cv2.LUT(g, lut)
g = cv2.imread('x.png', 0); out = gamma_correct(g, 0.4)
plt.subplot(2,2,1); plt.imshow(g, cmap='gray', vmin=0, vmax=255)
plt.subplot(2,2,2); plt.imshow(out, cmap='gray', vmin=0, vmax=255)
plt.subplot(2,2,3); plt.hist(g.ravel(), 256, [0,256])
plt.subplot(2,2,4); plt.hist(out.ravel(), 256, [0,256])
```

2B piecewise map; r=125 -> 30 + (195/90)*45 = 127.5; mid slope 2.17 > 1 boosts mid contrast, ends compressed:

```
out = np.interp(g, [0,80,170,255], [0,30,225,255]).astype(np.uint8)
```

2C video pipeline (equalizeHist after applyColorMap raises an assertion: needs CV_8UC1, colormap gives 3 channels):

```
def gray_world(img):
    b, gc, r = cv2.split(img.astype(np.float32))
    avg = (b.mean() + gc.mean() + r.mean())/3
    b *= avg/(b.mean()+1e-6); gc *= avg/(gc.mean()+1e-6)
    r *= avg/(r.mean()+1e-6)
    return np.clip(cv2.merge([b,gc,r]), 0, 255).astype(np.uint8)
glut = np.array([255*(i/255.0)**0.7 for i in range(256)], np.uint8)
cap = cv2.VideoCapture('echo.mp4')
while True:
    ret, f = cap.read()
    if not ret: break
    eq = cv2.equalizeHist(cv2.cvtColor(f, cv2.COLOR_BGR2GRAY))
    out = cv2.LUT(gray_world(cv2.applyColorMap(eq, cv2.COLORMAP_JET)), glut)
    cv2.imshow('Raw | Enhanced', np.hstack([f, out]))
    if cv2.waitKey(30) & 0xFF == ord('q'): break
    cap.release(); cv2.destroyAllWindows()
```

C3 Lab 3 tasks

3A translate (100,50), rotate 30 deg about centre at 0.8, show three. getRotationMatrix2D order = (center (x,y), angle, scale):

```
h, w = img.shape[:2]
T = np.float32([[1,0,100],[0,1,50]]); tr = cv2.warpAffine(img, T, (w,h))
M = cv2.getRotationMatrix2D((w/2,h/2), 30, 0.8); print(M)
ro = cv2.warpAffine(img, M, (w,h))
for i,(im,t) in enumerate([(img,'Orig'),(tr,'Translated'),(ro,'Rotated')],1):
    plt.subplot(1,3,i); plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
    plt.title(t); plt.axis('off')
```

3B rotate 45 deg without losing corners (bounding box of a rotated rectangle: new_w = w|cos|+h|sin|, new_h = w|sin|+h|cos|):

```
M = cv2.getRotationMatrix2D((w/2,h/2), 45, 1.0)
c, s = abs(M[0,0]), abs(M[0,1])
nw, nh = int(h*s + w*c), int(h*c + w*s)
# centre on new canvas
M[0,2] += nw/2 - w/2; M[1,2] += nh/2 - h/2
out = cv2.warpAffine(img, M, (nw,nh))
```

3C document scanner (affine fails: trapezoid has non-parallel sides; affine preserves parallelism):

```
def order_points(pts):
    pts = np.array(pts, np.float32)
    s = pts.sum(axis=1); d = np.diff(pts, axis=1).ravel()
    return np.float32([pts[np.argmax(s)], pts[np.argmin(d)], # tl, tr
        pts[np.argmax(s)], pts[np.argmax(d)]] # br, bl)
```

```
def scan(img, pts):
    src = order_points(pts); tl, tr, br, bl = src
    W = int(max(np.linalg.norm(tr-tl), np.linalg.norm(br-bl)))
    H = int(max(np.linalg.norm(bl-tl), np.linalg.norm(br-tr)))
    dst = np.float32([[0,0],[W-1,0],[W-1,H-1],[0,H-1]])
    M = cv2.getPerspectiveTransform(src, dst)
    return cv2.warpPerspective(img, M, (W,H))
```

3D by hand: [1 0 100; 0 1 50; 0 0 1] * (10,20,1) = (110,70). Point (10,0), R90 = [[0,-1],[1,0]]: rotate then translate: (0,10) -> **(100,60)**; translate then rotate: (110,50) -> **(-50,110)**. Order matters.

C4 Lab 4 tasks

4A convolution from scratch (valid). filter2D does correlation (no kernel flip); true convolution = flip kernel with cv2.flip(k,-1):

```
def conv_valid(img, k):
    H, W = img.shape; kh, kw = k.shape
    f = img.astype(np.float32); k = k.astype(np.float32)
    out = np.zeros((H-kh+1, W-kw+1), np.float32)
    for y in range(H-kh+1):
        for x in range(W-kw+1):
            out[y,x] = np.sum(f[y:y+kh, x:x+kw] * k)
    return out
ref = cv2.filter2D(img.astype(np.float32), -1, k); p = k.shape[0]//2
print(np.allclose(conv_valid(img,k), ref[p:-p, p:-p], atol=1e-3))
```

4B Sobel vs Canny thresholds. 30/90 = many edges incl. texture/noise (unwanted); 100/200 balanced; 200/250 only strongest, weak edges missing, edges fragmented:

```
b = cv2.GaussianBlur(cv2.imread('a.jpg',0), (5,5), 1.4)
gx = cv2.Sobel(b, cv2.CV_64F, 1, 0); gy = cv2.Sobel(b, cv2.CV_64F, 0, 1)
res = [('Orig', b), ('Sobel', np.uint8(np.clip(np.hypot(gx,gy), 0, 255)))]
for lo, hi in [(30,90), (100,200), (200,250)]:
    res.append((f'Canny {lo}/{hi}', cv2.Canny(b, lo, hi)))
for i,(t,im) in enumerate(res,1):
    plt.subplot(2,3,i); plt.imshow(im, cmap='gray'); plt.title(t); plt.axis('off')
```

4C salt-and-pepper page: median FIRST (removes isolated dots without blurring); Gaussian first would smear each dot into a blob; gradients amplify noise so denoise before Canny:

```
d = cv2.medianBlur(gray, 3)
d = cv2.GaussianBlur(d, (3,3), 0)
edges = cv2.Canny(d, 50, 150)
```

4D by hand patch [[10,20,30],[10,20,30],[80,90,100]]: box centre = 390/9 = **43.3**; Sobel-y = (80+180+100) - (10+40+30) = **280**.

C5 Lab 5 tasks

5A uneven-lighting document. Global fails where the shadowed side falls below T (text lost) or the bright side gets specks. Expect: bs=11 noisy/hollow, bs=31 clean, bs=101 global-like; C=10 whiter background, thinner strokes:

```
g = cv2.imread('page.jpg', 0); res = [('Orig', g)]
for T in (80,127,170):
    res.append((f'G{T}', cv2.threshold(g, T, 255, cv2.THRESH_BINARY)[1]))
res.append(('Otsu', cv2.threshold(g,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)[1]))
for bs in (11,31,101):
    for C in (2,10):
        res.append((f'bs{bs} C{C}', cv2.adaptiveThreshold(g, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, bs, C)))
```

5B red object. Tight mask fails because shaded/specular pixels have lower S or V (or shifted hue). Fix: lower S,V bounds, widen H, close holes:

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
m = cv2.bitwise_or(cv2.inRange(hsv, (0,100,80), (10,255,255)),
    cv2.inRange(hsv, (170,100,80), (179,255,255)))
obj = cv2.bitwise_and(img, img, mask=m)
tight = cv2.inRange(hsv, (0,200,200), (3,255,255)) # too restrictive
m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, np.ones((5,5), np.uint8))
```

5C watershed with fraction sweep (pick the fraction where count is stable and matches a visual count, ~0.4-0.6):

```
def watershed_count(img, frac):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, th = cv2.threshold(gray,0,255,cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
    k = np.ones((3,3), np.uint8)
    op = cv2.morphologyEx(th, cv2.MORPH_OPEN, k, iterations=2)
    bg = cv2.dilate(op, k, iterations=3)
    dist = cv2.distanceTransform(op, cv2.DIST_L2, 5)
    _, fg = cv2.threshold(dist, frac*dist.max(), 255, 0); fg = np.uint8(fg)
    unk = cv2.subtract(bg, fg)
    n, mk = cv2.connectedComponents(fg)
    mk = mk + 1; mk[unk == 255] = 0
    out = img.copy(); mk = cv2.watershed(out, mk)
    out[mk == -1] = (0,0,255)
    # n includes background
    return n - 1, out
for f in (0.2, 0.4, 0.6, 0.8): print(f, watershed_count(img, f)[0])
```

5D region growing (int) prevents uint8 wrap; seed matters because the criterion compares with the seed intensity):

```
from collections import deque
def region_growing(g, seed, thresh): # seed = (row, col)
    h, w = g.shape; mask = np.zeros((h,w), np.uint8)
    seen = np.zeros((h,w), bool); sv = int(g[seed])
    q = deque([seed]); seen[seed] = True
    while q:
        y, x = q.popleft()
        if abs(int(g[y,x]) - sv) < thresh:
            mask[y,x] = 255
            for dy, dx in ((1,0),(-1,0),(0,1),(0,-1)):
                ny, nx = y+dy, x+dx
                if 0 <= ny < h and 0 <= nx < w and not seen[ny,nx]:
                    seen[ny,nx] = True; q.append((ny,nx))
    return mask
for seed in [(120,80), (200,150)]:
    for t in (10,25,60): m = region_growing(g, seed, t)
```

5E design (fruits on shadowed plate): HSV inRange - assumes each fruit has a distinct hue; segments fruits well, shadows mostly change V not H; struggles with white/grey plate; key param hue range + S,V lower bounds. **K-Means (K=5-6, Lab)** - assumes colour clusters = regions; separates fruit/plate/background; the shadow may become its own cluster or similar fruits merge; key param K. **Watershed** - assumes blob cores; separates touching fruits; needs a clean

foreground mask first and over-splits textured skins; key param distance fraction. **Winner: HSV (+ morphology)**, because hue is robust to the shadow. Global/Otsu are poor here: shadows break one-threshold assumptions.

C6 Lab 6 tasks

6A lane detection (ROI removes buildings/sky; slope filter removes horizontal clutter; lower threshold = more false lines):

```
edges = cv2.Canny(cv2.GaussianBlur(gray,(5,5),0), 50, 150)
h, w = edges.shape; mask = np.zeros_like(edges)
roi = np.array([(0,h),(w//2-50,int(h*.6))),(w//2+50,int(h*.6))),(w,h)], np.int32)
cv2.fillPoly(mask, roi, 255); m = cv2.bitwise_and(edges, mask)
segs = cv2.HoughLinesP(m, 1, np.pi/180, 40, minLineLength=40, maxLineGap=100)
for x1,y1,x2,y2 in (segs[:,0] if segs is not None else []):
    if x2 == x1: continue
    sl = (y2-y1)/(x2-x1)
    if abs(sl) < 0.5: continue
    # y down: left lane sl<0
    col = (255,0,0) if sl < 0 else (0,0,255)
    cv2.line(img, (x1,y1), (x2,y2), col, 3)
```

6B coin count. (i) one coin, two circles -> raise minDist; (ii) real coins missed -> lower param2 (check radius range); (iii) bottle cap detected -> narrow minRadius/maxRadius or raise param2:

```
g = cv2.medianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 5)
c = cv2.HoughCircles(g, cv2.HOUGH_GRADIENT, 1, 30, param1=100, param2=30,
    minRadius=10, maxRadius=60)
if c is not None:
    for x, y, r in np.uint16(np.around(c))[0]:
        cv2.circle(img, (x,y), r, (0,255,0), 2); cv2.circle(img,(x,y),2,(0,0,255))
    print("Coins:", c.shape[1])
```

6C panorama. Swapping src/dst makes H map left->right, so the right image lands in the wrong place. RANSAC is needed because the ratio test still leaves wrong matches and one outlier ruins a least-squares fit:

```
def stitch_pair(left, right):
    g1 = cv2.cvtColor(left, cv2.COLOR_BGR2GRAY)
    g2 = cv2.cvtColor(right, cv2.COLOR_BGR2GRAY)
    sift = cv2.SIFT_create()
    kp1, d1 = sift.detectAndCompute(g1, None) # LEFT
    kp2, d2 = sift.detectAndCompute(g2, None) # RIGHT
    mt = cv2.BFMatcher(cv2.NORM_L2).knnMatch(d2, d1, k=2) # query=RIGHT
    good = [m for m,n in mt if m.distance < 0.75*n.distance]
    if len(good) < 10: raise RuntimeError("not enough matches")
    src = np.float32([kp2[m.queryIdx].pt for m in good]).reshape(-1,1,2)
    dst = np.float32([kp1[m.trainIdx].pt for m in good]).reshape(-1,1,2)
    H, _ = cv2.findHomography(src, dst, cv2.RANSAC, 5.0) # RIGHT -> LEFT
    hl, wl = left.shape[:2]; hr, wr = right.shape[:2]
    cr = np.float32([[0,0],[0,hr],[wr,hr],[wr,0]]).reshape(-1,1,2)
    cl = np.float32([[0,0],[0,hl],[wl,hl],[wl,0]]).reshape(-1,1,2)
    allp = np.concatenate([cl, cv2.perspectiveTransform(cr, H)])
    xmin, ymin = np.int32(allp.min(axis=0).ravel() - 0.5)
    xmax, ymax = np.int32(allp.max(axis=0).ravel() + 0.5)
    T = np.array([[1,0,-xmin],[0,1,-ymin],[0,0,1]], np.float64)
    pano = cv2.warpPerspective(right, T @ H, (xmax-xmin, ymax-ymin))
    pano[-ymin:-ymin+hl, -xmin:-xmin+wl] = left
    ys, xs = np.where(cv2.cvtColor(pano, cv2.COLOR_BGR2GRAY) > 0)
    return pano[ys.min():ys.max()+1, xs.min():xs.max()+1]
# 3+ images: pano = stitch_pair(pano, next_img) | quick: cv2.Stitcher_create()
```

6D SIFT object recognition (found = enough good matches AND enough RANSAC inliers; a false positive can come from repetitive texture or a low threshold):

```
ref = cv2.imread('obj.jpg', 0); h, w = ref.shape
# once
sift = cv2.SIFT_create(); kp1, des1 = sift.detectAndCompute(ref, None)
bf = cv2.BFMatcher(cv2.NORM_L2)
def find_object(scene):
    g = cv2.cvtColor(scene, cv2.COLOR_BGR2GRAY)
    kp2, des2 = sift.detectAndCompute(g, None)
    if des2 is None or len(kp2) < 2: return scene
    good = [m for m,n in bf.knnMatch(des1, des2, k=2)
        if m.distance < 0.75*n.distance]
    if len(good) >= 10:
        src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
        dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
        H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
        if H is not None and mask.sum() >= 10:
            box = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-1,1,2)
            cv2.polylines(scene, [np.int32(cv2.perspectiveTransform(box,H))],
                True, (0,255,0), 3)
    return scene
```

6E wavelet anomaly detection (threshold only DETAIL coeffs: noise/spikes live there, cA holds the true shape; residual = original - denoised is ~0 except at anomalies):

```
co = pywt.wavedec(sig, 'db4', level=3)
sigma = np.median(np.abs(co[-1]))/0.6745
# universal threshold
thr = sigma*np.sqrt(2*np.log(len(sig)))
new = [co[0]] + [pywt.threshold(c, thr, mode='soft') for c in co[1:]]
# trim odd-length extra
den = pywt.waverec(new, 'db4')[:len(sig)]
res = sig - den
anom = np.abs(res) > 3*np.std(res)
plt.plot(sig); plt.plot(den); plt.plot(np.where(anom)[0], sig[anom], 'ro')
```

6F by hand: pixel (30,40): theta=0 -> rho=30; theta=90 -> rho=40; theta=53.13 -> 18+32 = **50**. The vertical line x=30 passes through (theta=0, rho=30).

C7 Detecting PCs / monitors / keyboards

Known item (reference photo): SIFT + ratio + RANSAC; for several instances, delete keypoints inside each found box and repeat. Decide by min inliers (~12). Limits: needs similar look; textureless monitors give few keypoints. **Any monitor (no reference):** Canny -> dilate -> findContours -> approxPolyDP(0.02*perimeter) -> keep 4-sided, convex, area > min, aspect ratio 1.3-2.0; on/off = mean brightness inside quad. **Deep learning** (YOLO etc.) is the practical alternative (outside syllabus).

```
sift, bf = cv2.SIFT_create(), cv2.BFMatcher(cv2.NORM_L2)
def detect(scene, ref_kp, ref_des, hw, min_in=12, max_n=10):
    h, w = hw; boxes = []
    g = cv2.cvtColor(scene, cv2.COLOR_BGR2GRAY)
    kp, des = sift.detectAndCompute(g, None)
    cor = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-1,1,2)
    for _ in range(max_n):
        if des is None or len(kp) < 2: break
        # query=ref, train=scene
        pr = bf.knnMatch(ref_des, des, k=2)
        good = [a for a,b in (p for p in pr if len(p)==2)
                if a.distance < 0.75*b.distance]
        if len(good) < 10: break
        s = np.float32([ref_kp[m.queryIdx].pt for m in good]).reshape(-1,1,2)
        d = np.float32([kp[m.trainIdx].pt for m in good]).reshape(-1,1,2)
        H, mk = cv2.findHomography(s, d, cv2.RANSAC, 5.0)
        if H is None or int(mk.sum()) < min_in: break
        box = cv2.perspectiveTransform(cor, H); boxes.append(np.int32(box))
        keep = [i for i,k in enumerate(kp)
                if cv2.pointPolygonTest(box.reshape(-1,2), k.pt, False) < 0]
        kp = [kp[i] for i in keep]; des = des[keep] if keep else None
    return boxes

# rectangles (screens) without a reference:
edges = cv2.dilate(cv2.Canny(cv2.GaussianBlur(g, (5,5),0),50,150), np.ones((3,3),np.uint8))
cnts,_ = cv2.findContours(edges, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
for c in cnts:
    if cv2.contourArea(c) < 5000: continue
    a = cv2.approxPolyDP(c, 0.02*cv2.arcLength(c, True), True)
    x, y, w_, h_ = cv2.boundingRect(a)
    if len(a)==4 and cv2.isContourConvex(a) and 1.3 < w_/h_ < 2.0:
        cv2.polylines(img, [a], True, (0,255,0), 3)
```